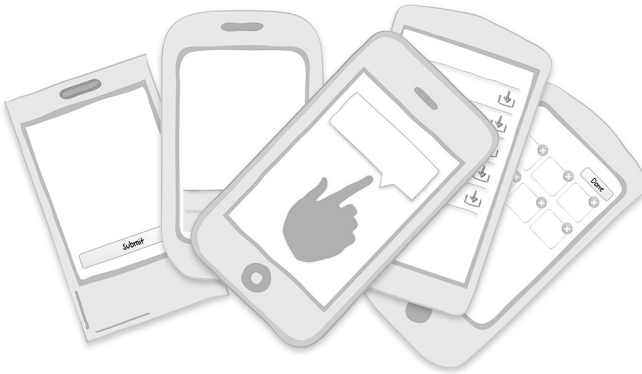


Tools



Patterns

Toolbar, Toolbox, Call to Action Button, Inline Actions, Multi-State Buttons, Contextual Tools, Bulk Actions, Lock Screen Controls

In Chapter 1, we looked at how screen size constraints are forcing designers to think off-canvas and explore new solutions. Before diving into this chapter, I encourage you to embrace another challenge—shed three decades of abstract desktop metaphors and start designing, really designing, for touch interfaces. For inspiration, start with Josh Clark’s “Buttons Are a Hack” campaign (<http://globalmoxie.com/blog/buttons-are-a-hack.shtml>):

Buttons are a hack. As in the real world, they’re often necessary, but they work at a distance—secondary tools to work on primary objects. A light switch here turns on a lightbulb there. These indirect interactions must be learned; they’re not contextually obvious. The revolution that touchscreen devices are working is that they allow us, more and more, to use primary content as a control, to create the illusion of direct interaction.

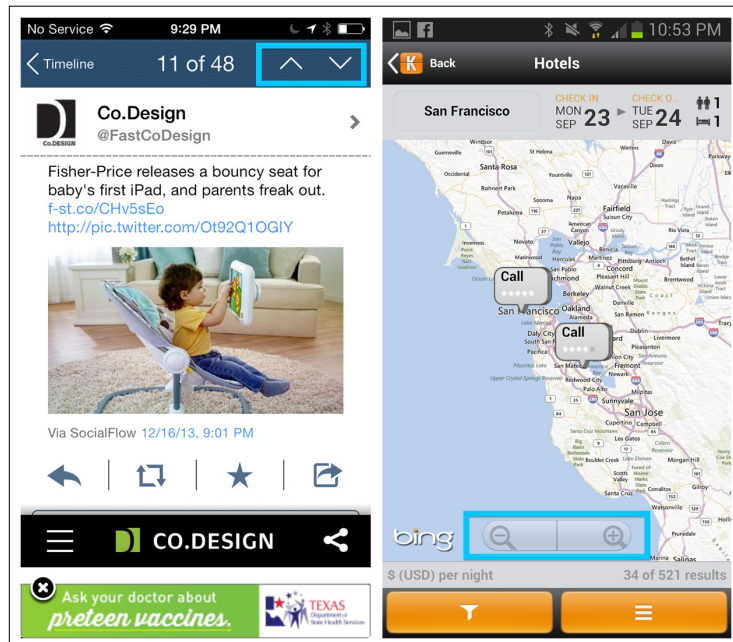
I don't mean to suggest that we throw out all of our familiar buttons entirely. Light switches shall remain necessary, after all, and so shall buttons, especially where it's necessary to trigger abstract actions ("share via Twitter," for example). But it's important to recognize those devices for what they are: necessary hacks for moments when direct interaction isn't possible. Touchscreen interfaces allow that direct interaction in many more contexts. As new solutions arise, we should be open to putting our time-tested workarounds aside. When designing an interaction for touch, always ask: do I really need another button or control for this?

Let's look at examples from TweetCaster and Kayak. In TweetCaster, the main interface is a list of tweets. Tapping one opens it in a new screen. To navigate to the previous or next tweet, you use two buttons (up and down arrows) in the navigation bar. But do we really need buttons for navigating to next or previous items, or is this just a holdover from desktop design? Page Swiping is a more efficient and natural way to navigate content on a touchscreen, as you'll see when we look at that pattern later in this chapter.

And what about the plus/minus magnifying glasses on the Kayak map? I think I remember those from my first computer in the '90s. Ditch the zoom buttons, people—the pinch-to-zoom gesture's got this covered.

FIGURE 5-1.

TweetCaster for iOS
and Kayak for Android:
outdated desktop
metaphors



I'm fairly certain that by the time the next edition of this book comes out, some new, creative, touch-based tool patterns will have emerged. For now, we'll look at the most common ones today. True, some of these patterns are versions of the old desktop metaphors—they're simply triggered by a touch instead of a click. But others incorporate touch to respond to users in ways that desktop interfaces can't.

Toolbar

The Toolbar is one of the most common ways to provide screen-specific actions. Each operating system has its own terminology and guidelines for the Toolbar.

iOS

The iOS Human Interface Guideline (<http://bit.ly/NoLrna>) says:

Although a toolbar looks similar to a navigation bar or a tab bar, it doesn't enable navigation. Instead, a toolbar gives users controls that act on the contents of the current screen. It always appears at the bottom edge of a screen or view on iPhone.

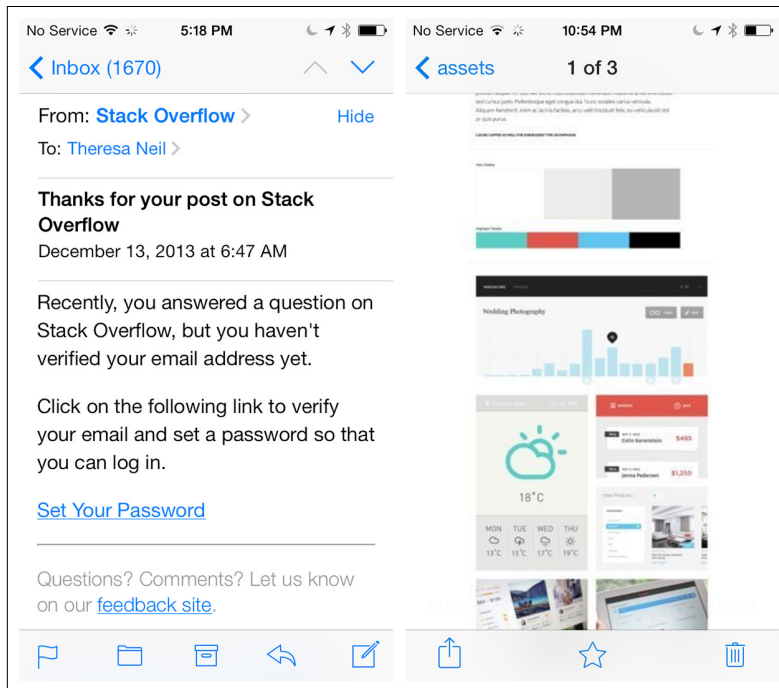


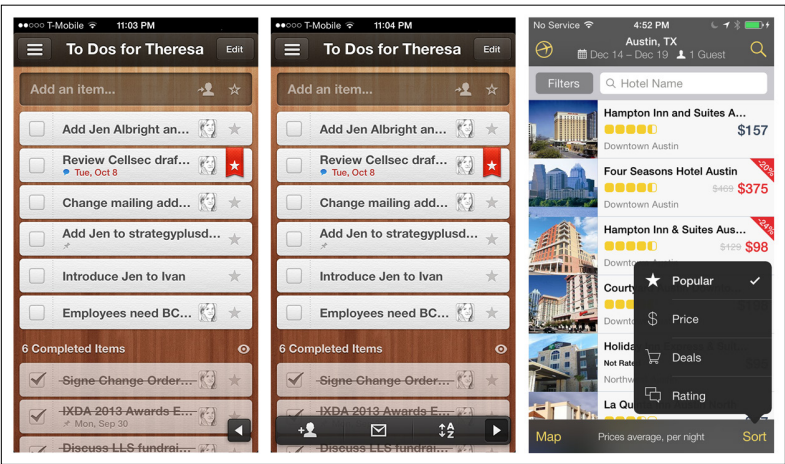
FIGURE 5-2.

Mail and Dropbox for iOS: don't confuse Toolbars with navigation bars

Expedia has created a cascading menu from the base Toolbar, and Wunderlist designed a custom collapsing Toolbar to save real estate.

FIGURE 5-3.

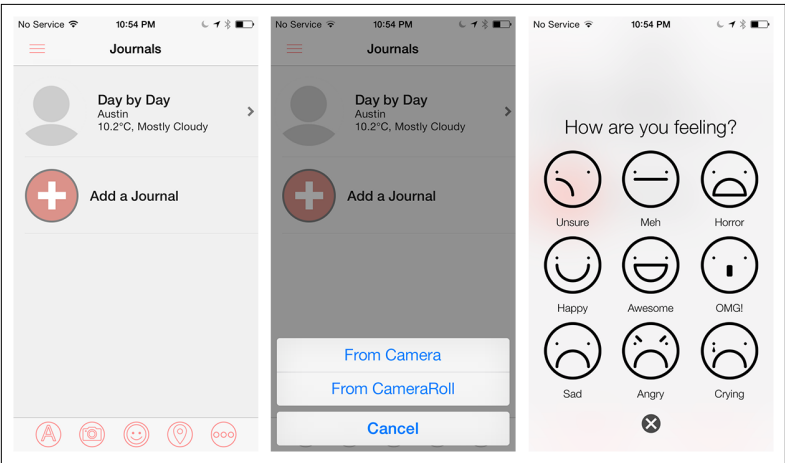
Expedia and Wunderlist
Custom Toolbars



Alternatively, an action button may launch an Action Sheet, which displays additional options for the selected button. In Narrato, for instance, the photo button opens a standard Action Sheet for choosing the photo source, while the smiley face button opens a custom Action Sheet for selecting a mood. This is an example of a smart design that stays within the spirit—if not the letter—of the iOS Design Guide.

FIGURE 5-4.

Narrato for iOS: two
different Action Sheet
designs



Android

The Jelly Bean release brought some changes to the Android interface (<http://developer.android.com/design/patterns/actionbar.html>). There is not a dedicated Toolbar, as in iOS. Instead, action buttons are assigned to the top right of the Action Bar, a dedicated piece of real estate at the top of each screen that is generally persistent throughout the app.

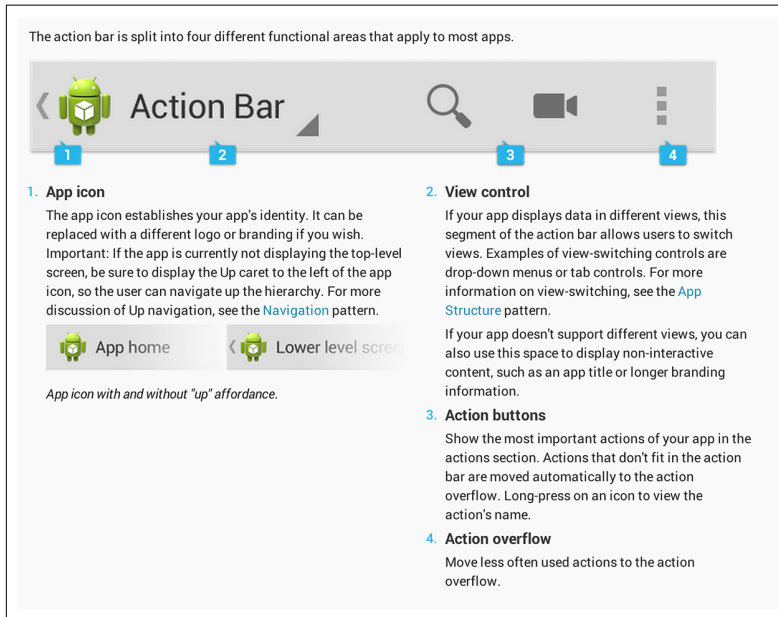


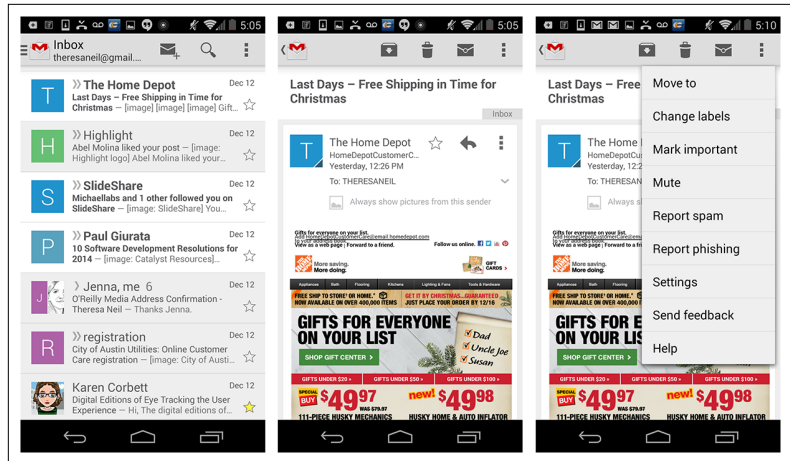
FIGURE 5-5.

The Action Bar is split into four functional areas that apply to most apps

The action buttons should be specific to the current page context, like in Gmail. Primary actions should be visible in the Action Bar at the top, but secondary actions belong in the Overflow Menu, which can be accessed via the vertical ellipsis icon.

FIGURE 5-6.

Gmail for Android:
ActionBar buttons are
determined by screen
context



One of the nice features of Android is that a long press on an action button brings up a tooltip explaining what the button is for. One of the not-so-nice features is that the Action Bar has to hold any necessary navigation controls (title, drawer icon, up button, spinner, etc.), as well as the action buttons, so it can get crowded very quickly.

Windows Phone

The Toolbar in Windows Phone apps is called the Application Bar (<http://bit.ly/NoMtPY>). The primary actions should be displayed as labeled icons, and secondary actions (called Menu Items) can be tucked under the ellipsis icon.

I was surprised to see that Amazon is trying to make the Application Bar a primary navigation control (see the Home, Search, and Cart icons at the bottom of every screen). It makes me think that the product design was driven by iOS aficionados, rather than a team familiar with Windows Phone design paradigms.

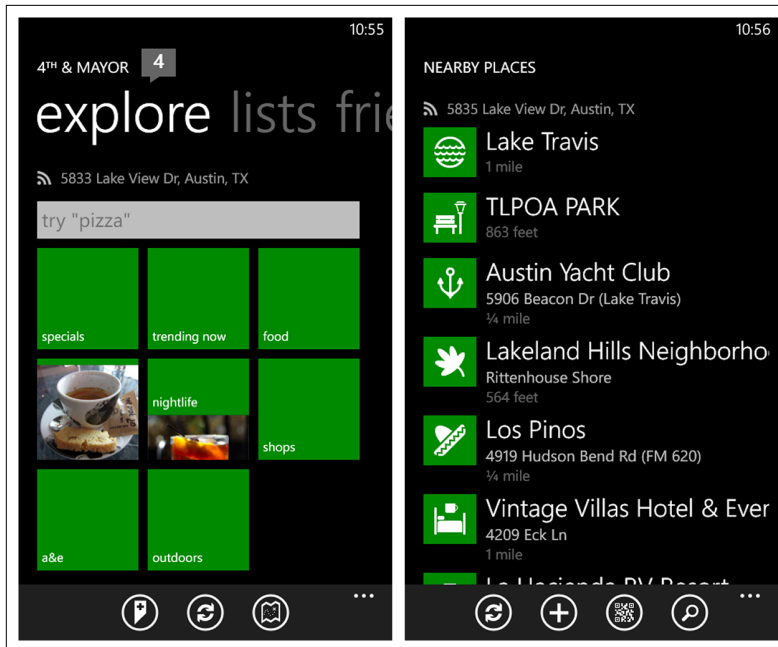


FIGURE 5-7.

4th & Mayor for Windows Phone: the Application Bar is for primary actions

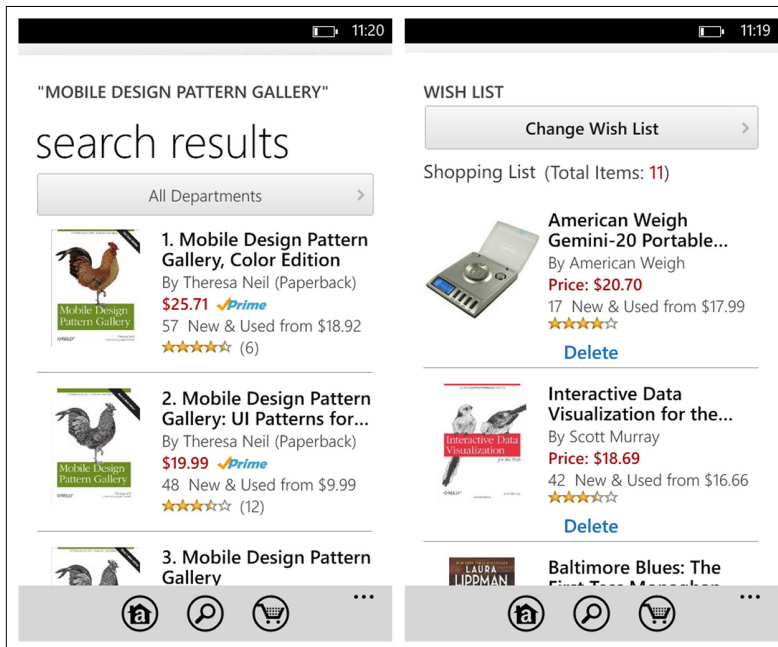


FIGURE 5-8.

Amazon for Windows Phone: incorrectly using the Application Bar for primary navigation

Like Android, Windows Phone also offers a way to expose the icon labels. Tapping the ellipsis icon heightens the Application Bar just a bit to reveal the labels below the icons.



FIGURE 5-9.

Windows Phone
Application Bar:
tapping the ellipsis
reveals labels

[NOTE]

Follow OS-specific recommendations for Toolbar placement and behaviors. Choose icons that are familiar and easy to recognize.

OS-Neutral Pattern: Contextual Toolbar

A Contextual Toolbar can also serve the action buttons for a text field or text area, like the formatting and attachment options shown in the Twitter, Evernote, and OneNote Toolbars.

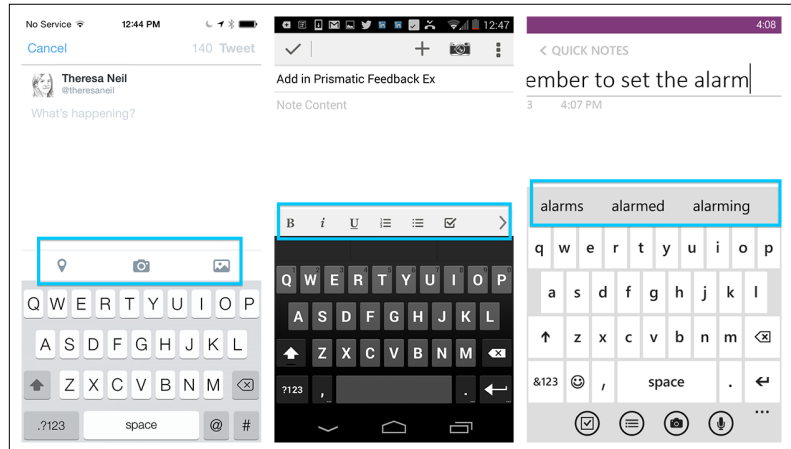


FIGURE 5-10.

Twitter for iOS,
Evernote for Android,
and OneNote for
Windows Phone:
Contextual Toolbars for
text entry actions

Toolbox

A Toolbox is like a Toolbar on steroids. You'll see this pattern most often in photography and design apps that offer a lot of tools for effects and treatments.

Typically, the main Toolbox is positioned across the bottom of the screen. The options may exceed the screen width and scroll off the screen. Selecting an option from the main Toolbox can reveal a new set of options specific to the selected tool. In Pixlr, for example, selecting Adjustment shows all the possible Adjustment options, like Whiten, Touch Up, Focal Blur, and more.



FIGURE 5-11.

Pixlr for Android: tool options are shown when a tool is selected from the Toolbox

Repix and Oggl use the same pattern, but instead of an overlay menu of options, a second row of tools is displayed to show the selected tool's options.

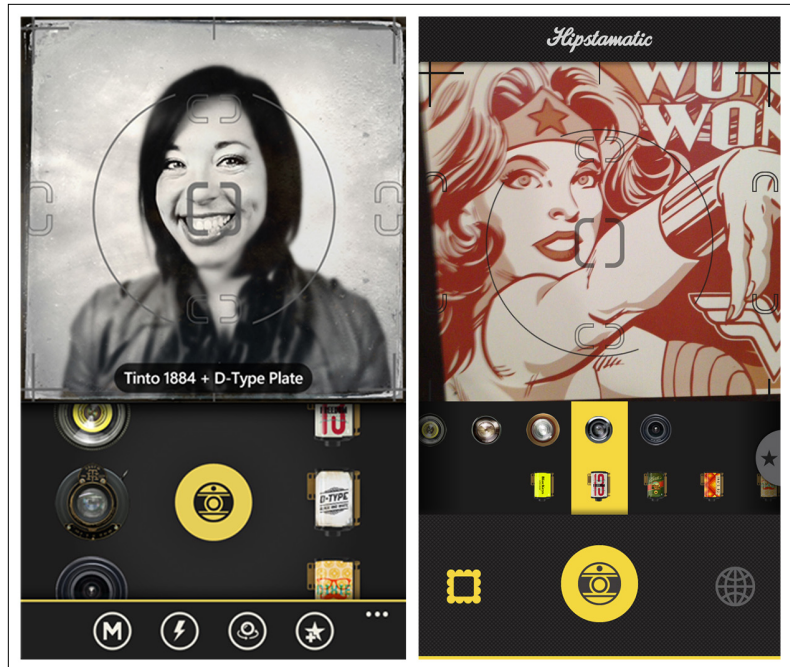


FIGURE 5-12.

Repix for iOS: tool options appear as a second row above the Toolbox

FIGURE 5-13.

Oggi for Windows
Phone and iOS: Toolbox
design varies with OS



Another implementation shows the primary set of tools and, once one is selected, reveals the suboptions for that tool, hiding the primary options. In the example from Aviary, orientation is selected first, and then the orientation options are displayed. If you use this pattern, keep two things in mind: it's best to restate which menu item is in effect since the main Toolbar is now out of sight, and you should offer an escape from this mode, such as a Cancel button.

[NOTE]

Labels plus icons improve the usability of Toolbars and Toolboxes. If tool options hide the main Toolbox, use some means (like a screen label) to let users know where they are, and be sure to include a Cancel button.

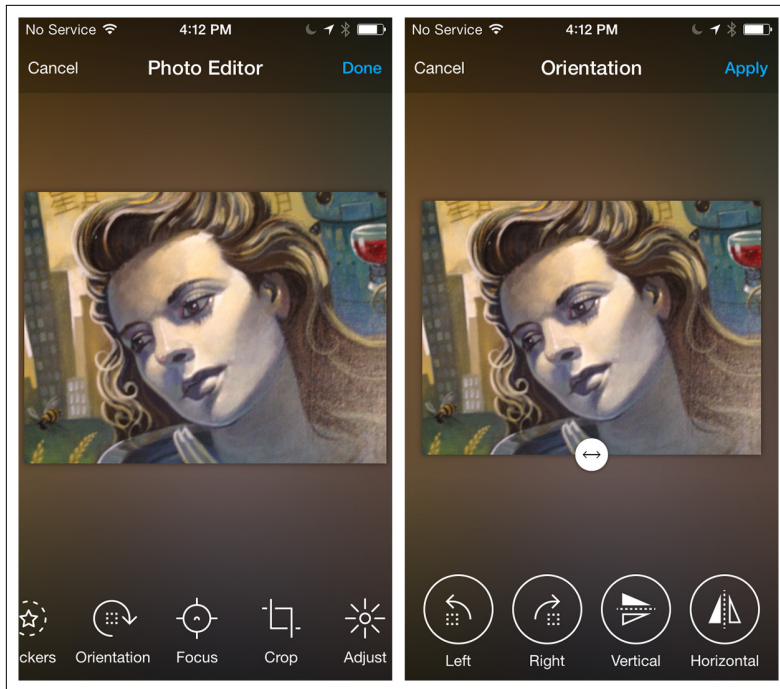
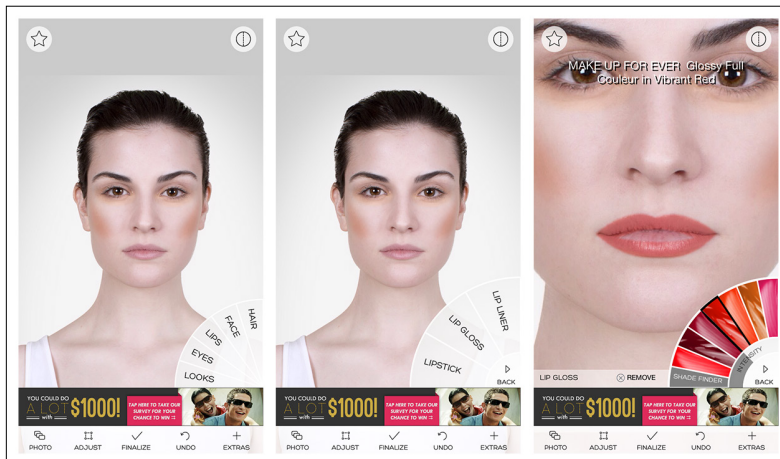


FIGURE 5-14.

Aviary for iOS: options
for selected tool
(orientation) are shown
on right

Virtual Makeover uses this pattern to present many different makeup tools. Its pie-style version of the Toolbox is surprisingly easy to navigate.

**FIGURE 5-15.**

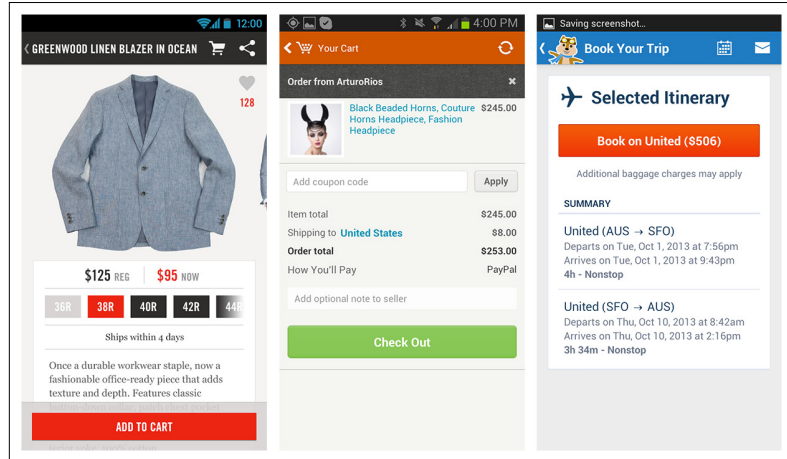
Virtual Makeover for iOS: pie-style Toolbox

Call to Action Button

Sometimes a Call to Action (CTA) Button may be a better option than a Toolbar when you are presenting a single action on the screen. Frank & Oak, Etsy, and Hipmunk are examples.

FIGURE 5-16.

Frank & Oak, Etsy, and Hipmunk for Android: prominent CTA Buttons



This pattern may also work for screens with one primary and one ancillary CTA, as in Foursquare and Fancy. Just make sure to differentiate the primary CTA visually from the other buttons.

In those examples, the CTAs are screen-specific and, more importantly, flow-specific. Over the years, designers have struggled with where and how to position a global CTA Button; that is, one that is always accessible throughout the app. In Chapter 1, we looked at variations of the standard Tab Bar that included a single Action Button in the middle of the bar, like Instagram and RunKeeperPro.

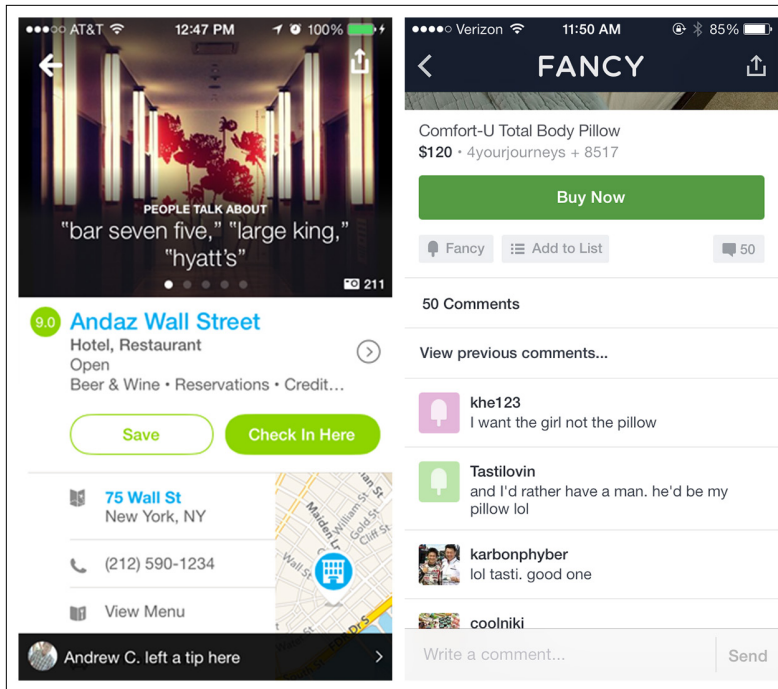


FIGURE 5-17.

Foursquare and Fancy for iOS: primary CTA is visually different from secondary CTAs

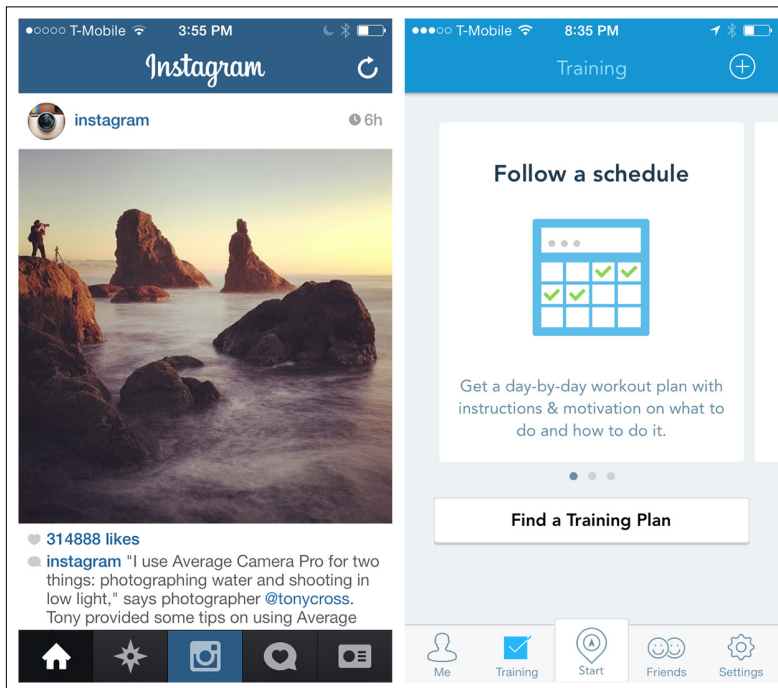
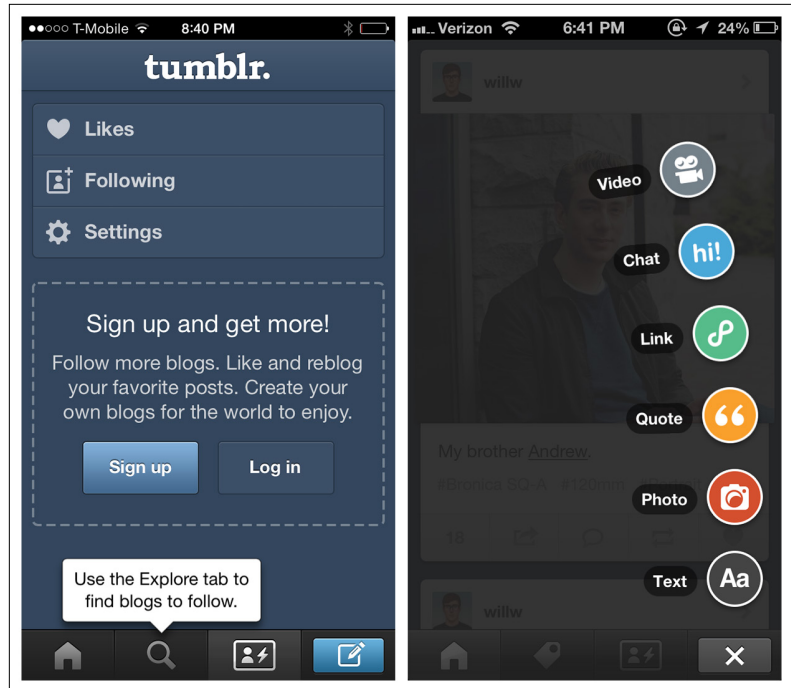


FIGURE 5-18.

Instagram and RunKeeperPro for iOS: global CTAs in Tab Bar

FIGURE 5-19.

Tumblr for iOS 6:
unbalanced right-side
global CTA Button with
flyout options



The Tumblr app for iOS 6 revealed a cascading menu of options from the global CTA Button. However, in the iOS 7 design, the global CTA Button opens a full screen of options.

One popular trend is a floating CTA Button positioned at the bottom of the screen. Foursquare, Sphere, and Rove's floating CTA Buttons are examples.

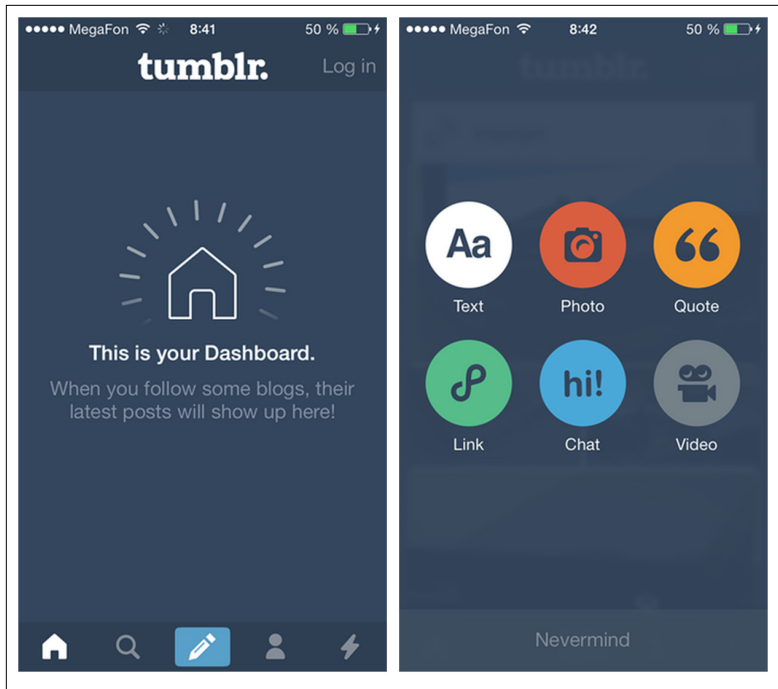


FIGURE 5-20.

Tumblr for iOS 7 centers the global CTA Button, and presents a full screen of action options

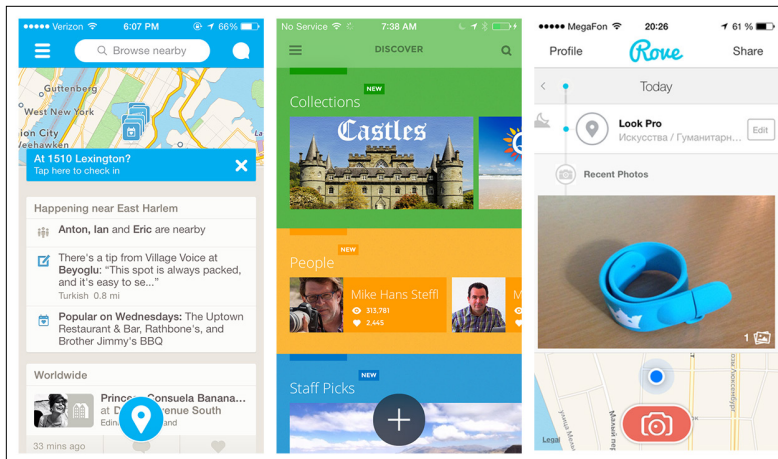


FIGURE 5-21.

Foursquare, Sphere, and Rove for iOS: floating CTAs

Path, Myspace, and QuickBooks reveal additional options when the floating CTA Button is tapped. In all three, the CTA Button shifts from a + to an x. By the way, this is the proper way to design symmetrical interactions (e.g., tap to open and tap to close in the same location).

[NOTE]

Consider using the CTA Button to help users complete a flow successfully, or quickly engage with applications that have a specific global driving action (like taking photos, sharing, or recording entries).

FIGURE 5-22.

Path for Android: when tapped, the floating CTA reveals floating options

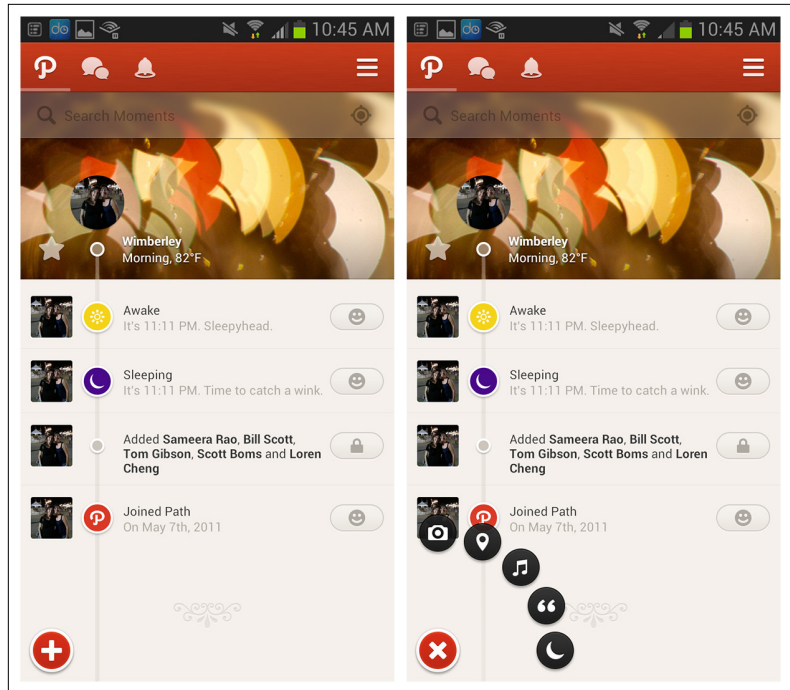
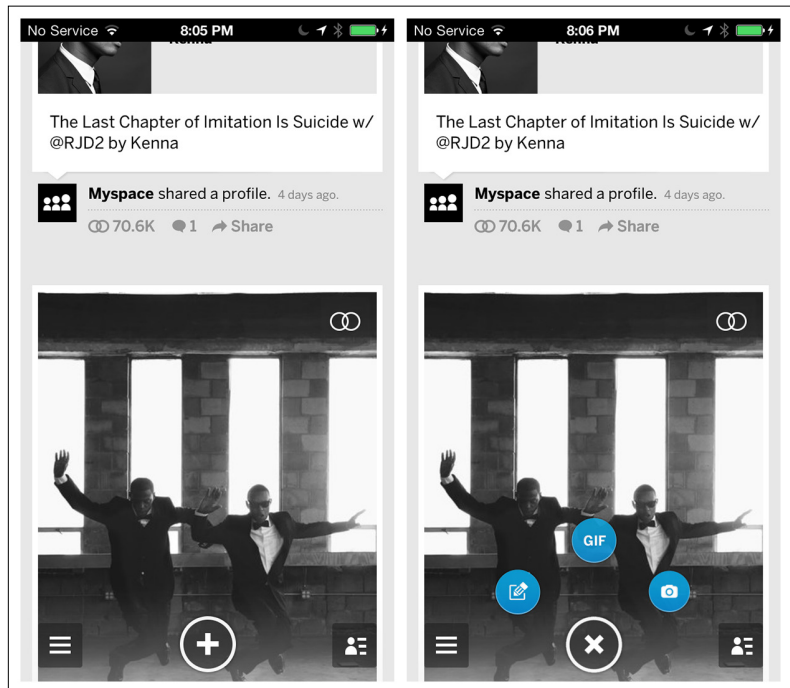


FIGURE 5-23.

Myspace for iOS: note that the CTA icon changes to an x (close) in its opened state



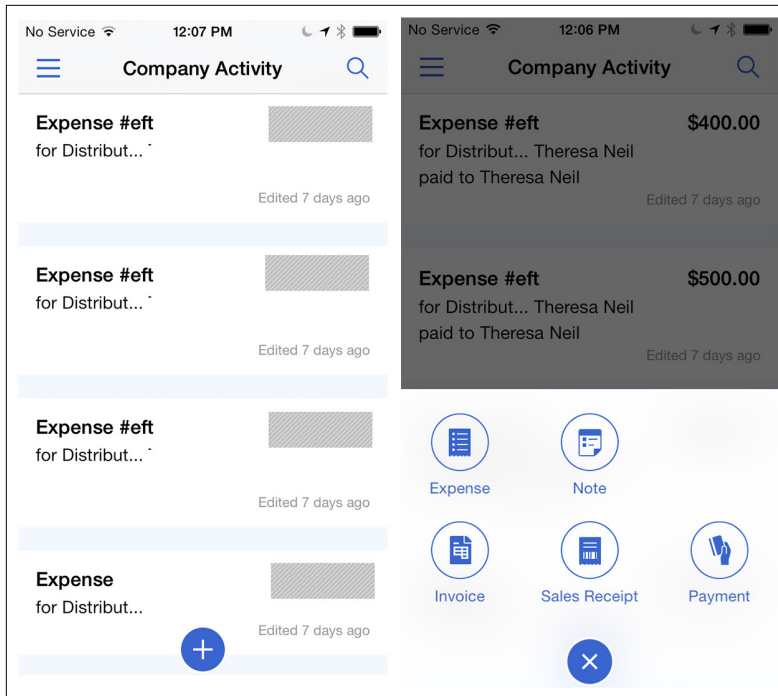


FIGURE 5-24.

QuickBooks for iOS:
another variation of
the open/close floating
CTA

Inline Actions

Inline Actions refer to action buttons that are inline with the object they affect, as opposed to being at a global or screen level. In The Nearby and Foodspotting, for example, each item in the list has an Inline Action button that can show two states (follow/following).

Many applications rely on stateful buttons for the Inline Actions, like the star buttons that can be toggled on and off in RetailMeNot and Gmail. See the next pattern, the Multi-State Button, for incorporating more than two states.

FIGURE 5-25.

The Nearby and Foodspotting for iOS: Inline Action buttons that can show two states

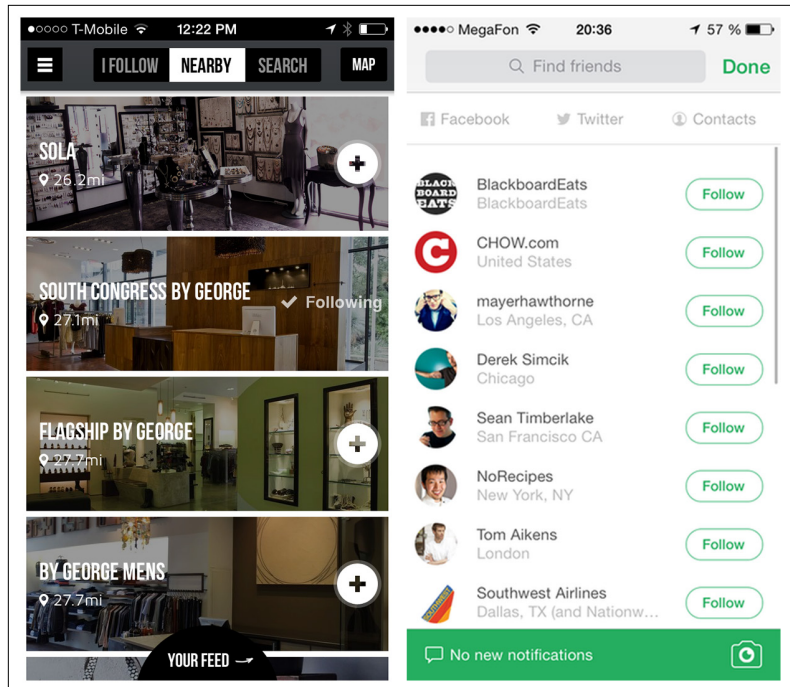
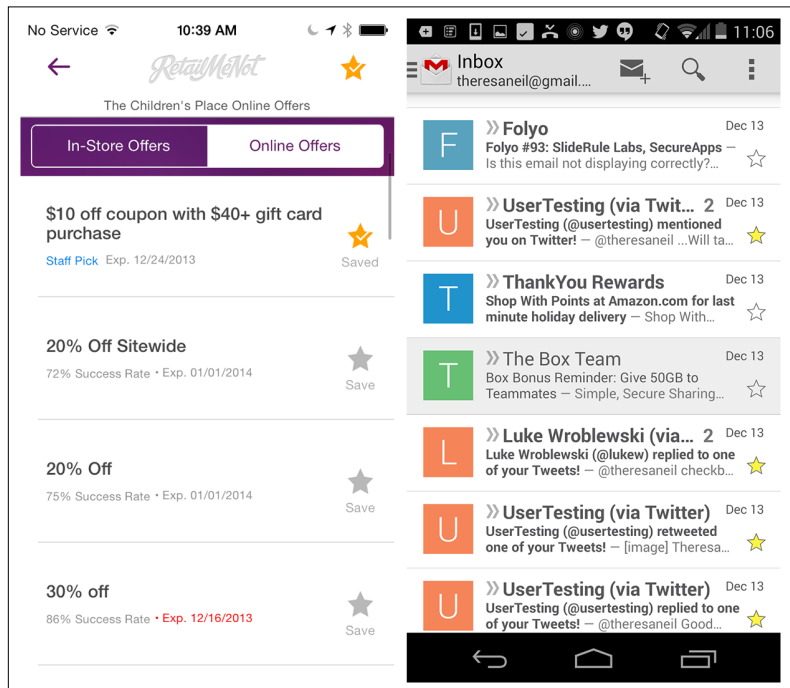


FIGURE 5-26.

RetailMeNot for iOS and Gmail for Android: stateful star buttons



The examples from LinkedIn and Facebook have two Inline Actions per row. If you want to display more than two actions per row, consider one of the Reveal patterns discussed later in this chapter.

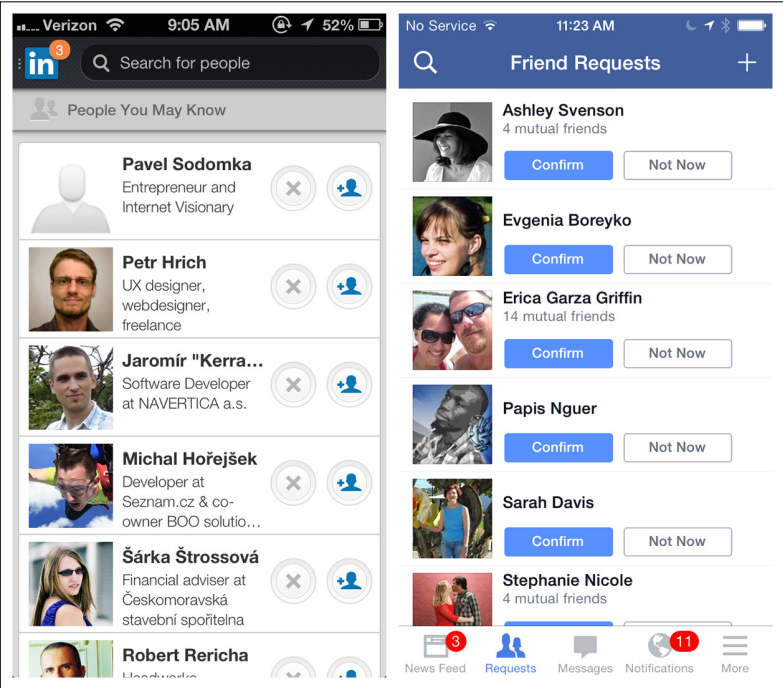
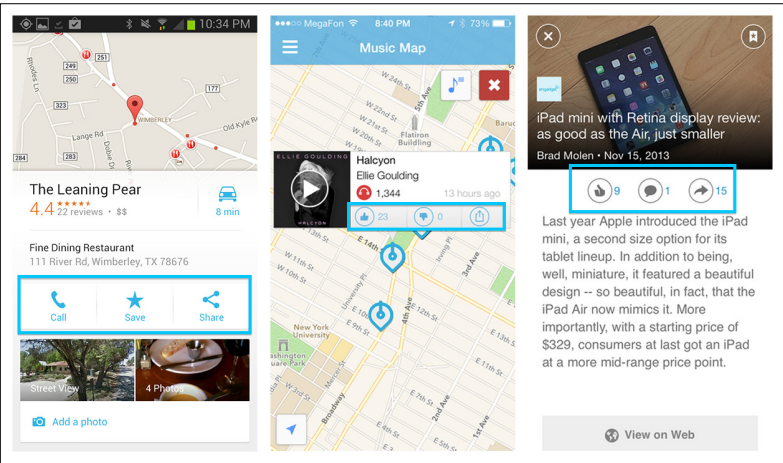


FIGURE 5-27. LinkedIn and Facebook for iOS: two Inline Actions per row

Google Maps, Soundwave, and Pulse provide Inline Actions at the object detail view.



[NOTE]
Inline Actions should be near the actionable object. Choose familiar icons and consider adding a text label.

FIGURE 5-28. Google Maps for Android, and Soundwave and Pulse for iOS: Inline Actions in object detail views

Multi-State Button

Normally, you shouldn't assign multiple actions to a UI control—it can overload the screen and the user. The Multi-State Button is an exception. This type of button acts as both an action trigger and a feedback mechanism.

For example, take the experience of downloading an app in the App Store for iOS. First you'll see the Inline Action to buy. Tap it again to install. Then it becomes an indicator for the install progress, and finally, once the app has downloaded, the button shows the Open option. See Chapter 9 for more examples.

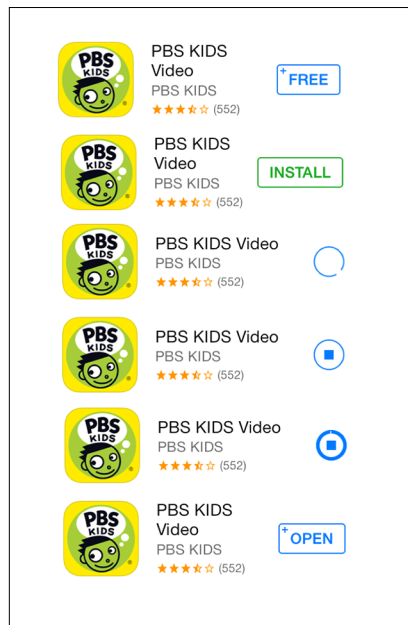


FIGURE 5-29.

App Store for iOS:
Multi-State Buttons
show buy, install,
progress, and open
options

[NOTE]

Multi-State Buttons work well for a series of tightly correlated processes that will be performed in rapid succession. Also consider using them instead of a dialog to prompt for confirmation before an irreparable action.

Another common use case for a Multi-State Button is a delete control. In many cases, popping open a delete confirmation dialog will break the user's flow (see Idiot Boxes in Chapter 11). But it makes sense to prompt for confirmation before completing an irreparable action, so the Multi-State Button serves nicely. (By the way, archiving an email is not an irreparable action, but emptying the trash is.)

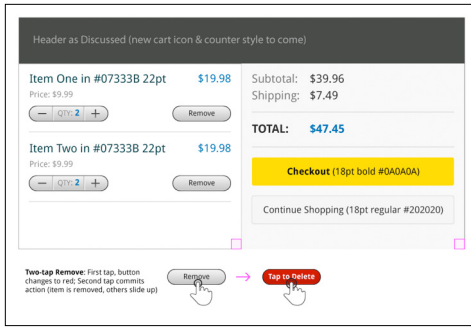


FIGURE 5-30. Wireframe and interaction notes from an Android application with a Multi-State Button for delete

Contextual Tools

Contextual Tools are specific to an object or task. It makes sense (and declutters the interface) if we reveal these tools only after the context has been established.

For example, tapping the ellipsis button on any of the pictures in Luvocracy reveals the Contextual Tools for the photo. This design is far more discoverable than the long-press-and-slide gesture Pinterest implemented. Of course, Pinterest looks cool with the animation, but watch the video of someone trying to use it for the first time.

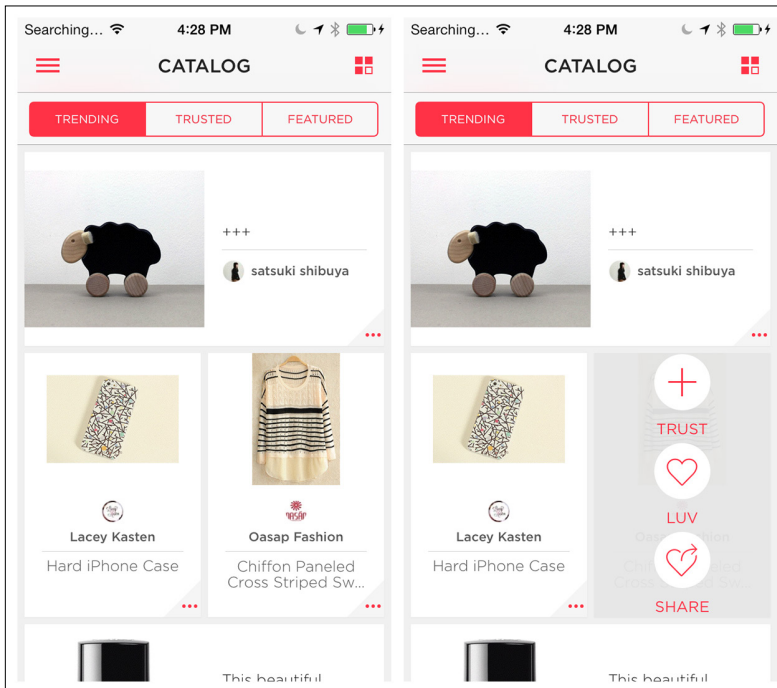
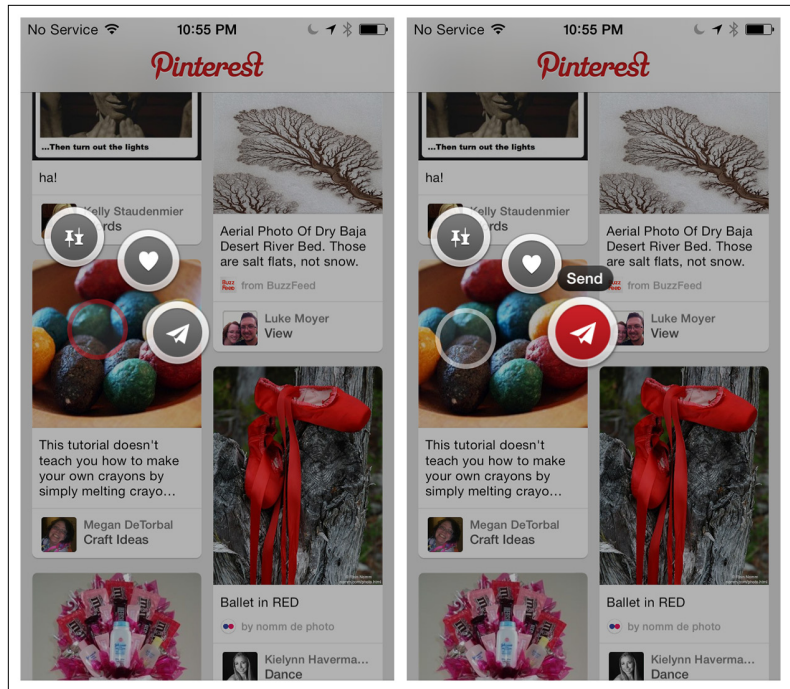


FIGURE 5-31. Luvocracy for iOS: tapping ellipsis reveals Contextual Tools

FIGURE 5-32.

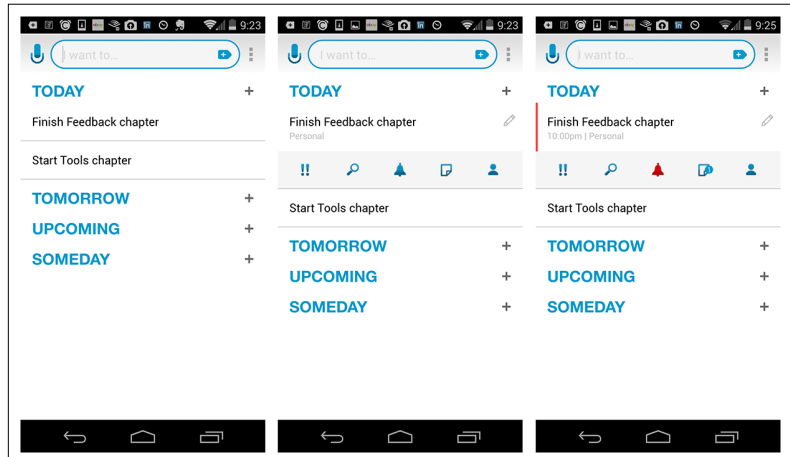
Pinterest for iOS:
long-press a Pin,
then hold and slide
to a button ([http://
www.youtube.com/
watch?v=dvXYztMT-bl](http://www.youtube.com/watch?v=dvXYztMT-bl))



Any.do also uses a simple tap to reveal Contextual Tools including set priority, add reminder, and share.

FIGURE 5-33.

Any.do for Android: tap
to reveal Contextual
Tools



What if the rows are already touch targets, like the folders in Dropbox? One idea is to add a “more” icon that, when tapped, will reveal the Contextual Tools. Dropbox uses an overlay to display the options, but you could also go with an inlay that pushes the other rows down.

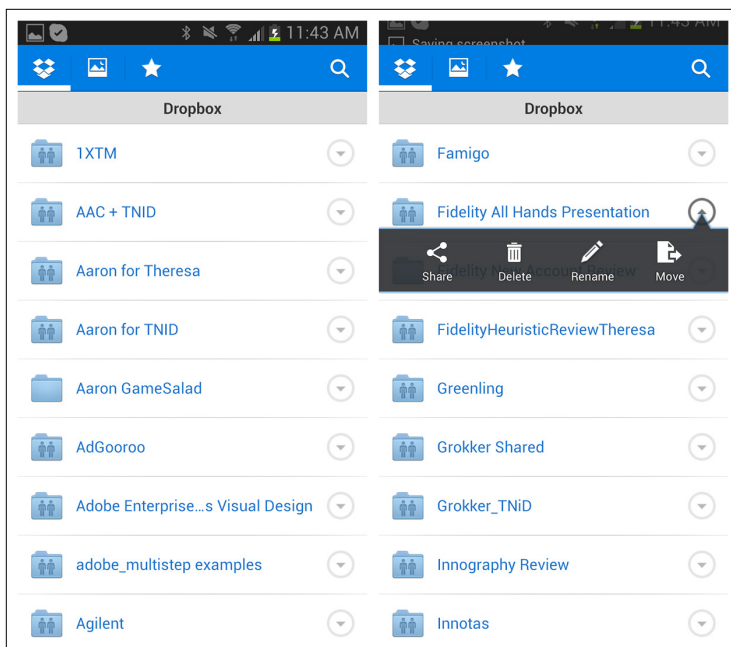


FIGURE 5-34.

Dropbox for Android:
tap the “more” icon to
reveal the Contextual
Tools overlay

Another option is to reveal the tools with a gesture other than tapping, like swiping. Flixster and BaconReader reveal Contextual Tools on horizontal swipe.

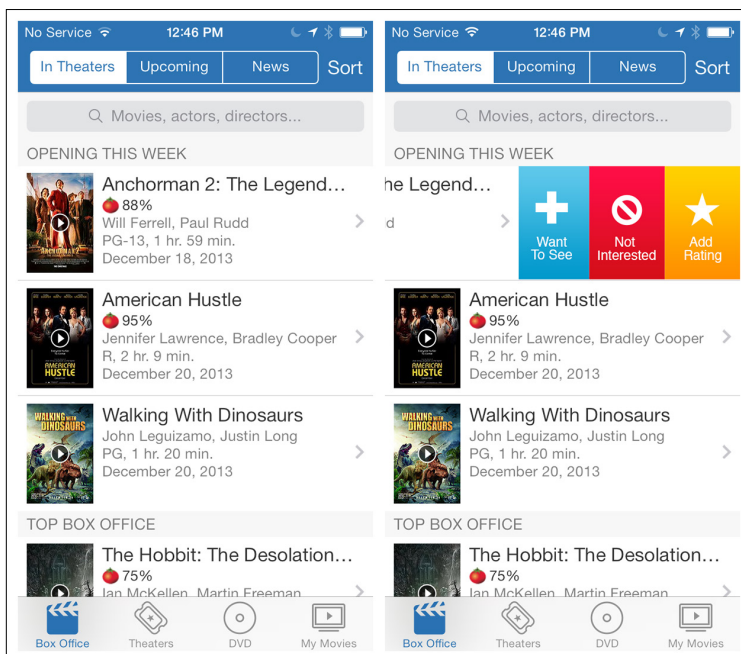
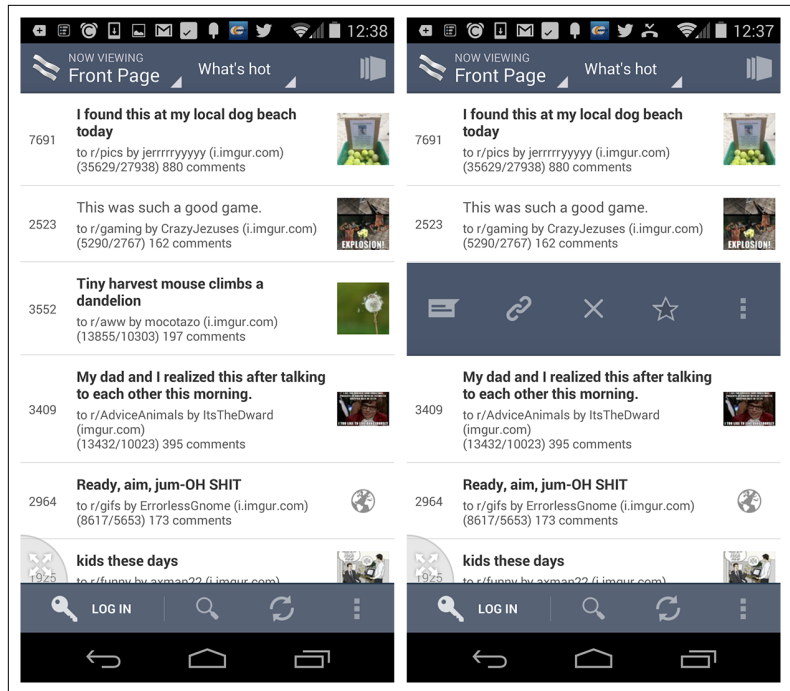


FIGURE 5-35.

Flixster for iOS:
swipe left to reveal
Contextual Tools

FIGURE 5-36.

BaconReader for
Android: swipe left
or right to reveal
Contextual Tools



This approach is very popular in productivity tools, like email and task apps. But beware—it is not as easy as it seems to design an intuitive and efficient gesture-based interface.

A long press can also be used to select an object and present Contextual Tools for it. In Box, long-pressing a file triggers the selection mode with the pressed object already selected. More objects can be selected, and the Contextual Tools (download, share, copy, delete, etc.) replace the Action Buttons. See the next pattern, Bulk Actions, for alternate ways to select multiple objects.

[NOTE]

When you choose a gesture to reveal Contextual Tools, start with the simplest solution first and test for discoverability and user efficiency.

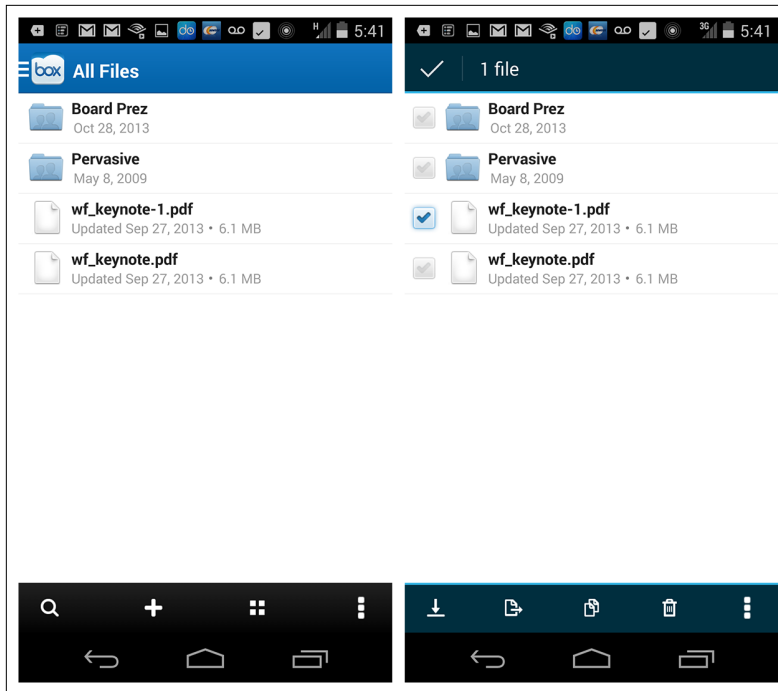


FIGURE 5-37.

Box for Android:
long-pressing a file
selects it and presents
Contextual Tools; other
items can be selected,
too

Bulk Actions

Common Bulk Actions include selection, deletion, and reordering. Instead of cluttering the main screen with all of these options listed for every item, provide a mode for Bulk Actions.

The native Photos app for iOS offers a “selection” mode for choosing photos from the camera roll to share. Once an item is selected, the Contextual Tools (share and delete) are revealed, and more items can be selected.

The iOS Stocks app offers an edit mode for reordering, deleting, and sorting stocks. Per iOS design standards, tapping Done in the top-right corner saves the changes and exits edit mode.

FIGURE 5-38.

Photos for iOS:
Contextual Tools
enable Bulk Actions for
all selected items

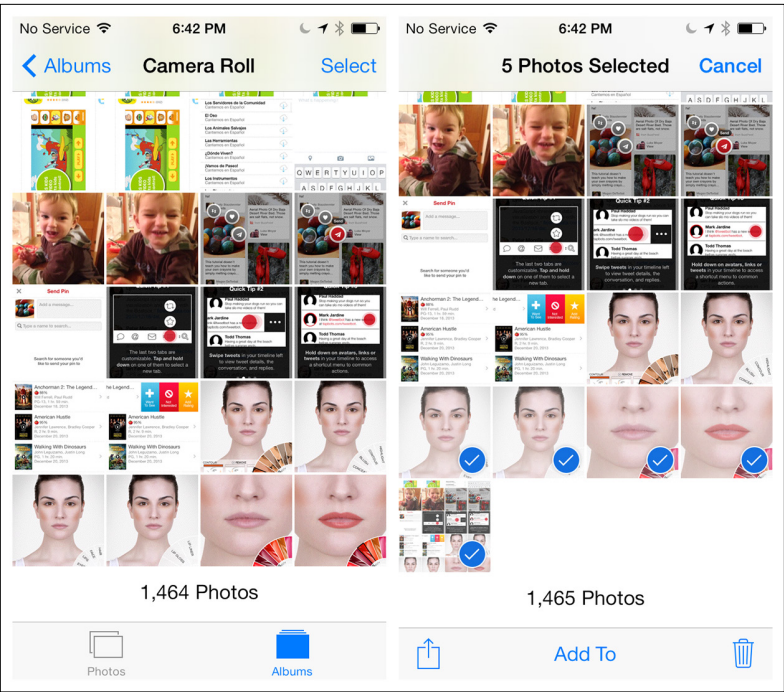
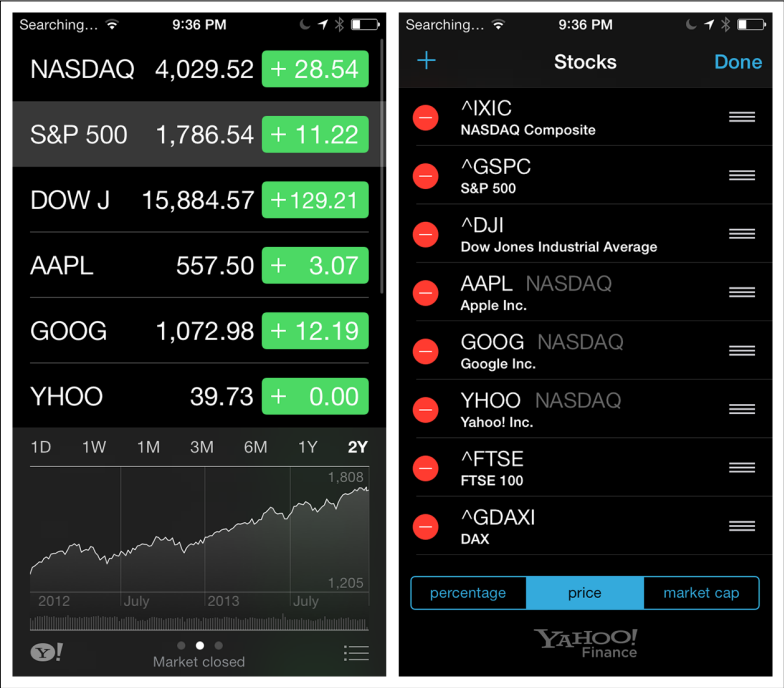


FIGURE 5-39.

Stocks for iOS: Bulk
Actions in edit mode



Fidelity for Android uses the same pattern, both for selecting stocks and for selecting and ordering the columns displayed in the Watch List.

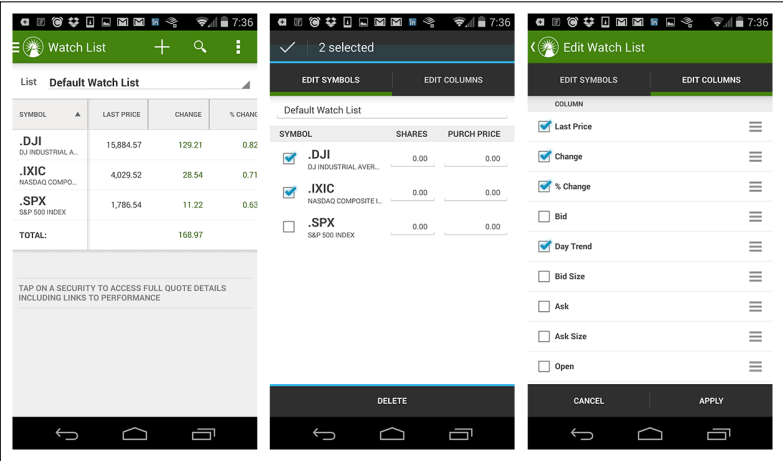


FIGURE 5-40.
Fidelity for Android:
bulk selection of stocks
and Watch List criteria

Path for iOS also offers Bulk Actions in edit mode, like add, remove, and assign a contact to your inner circle (the star icon).

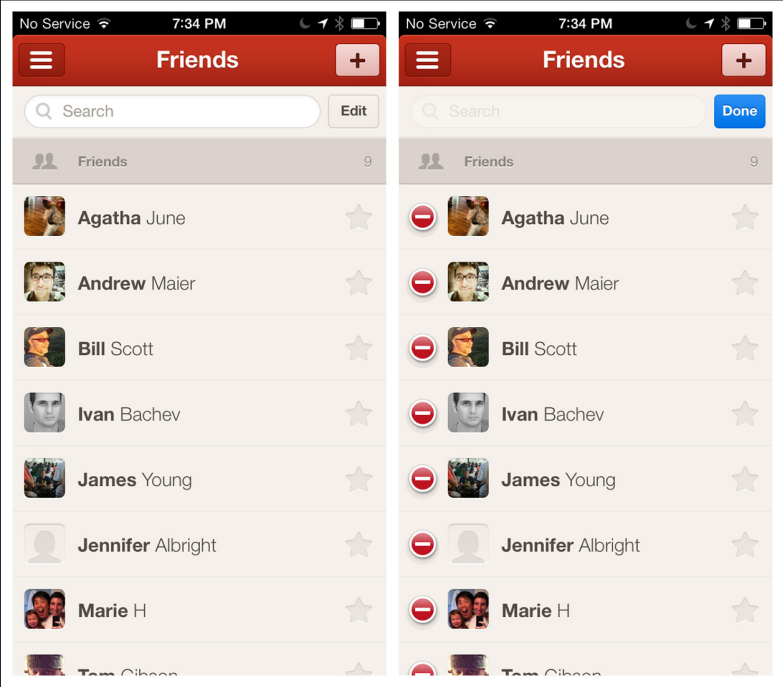


FIGURE 5-41.
Path for iOS: Bulk
Actions in edit mode
can act on multiple
items at once

[NOTE]

Bulk Actions like delete and reorder are best handled in a discrete edit mode. Provide an obvious option for exiting the mode.

To trigger this mode in Path for Android, you long-press any friend in the list. Path could improve this UI design by following Android design conventions and showing the number of selected items, instead of just shading them.

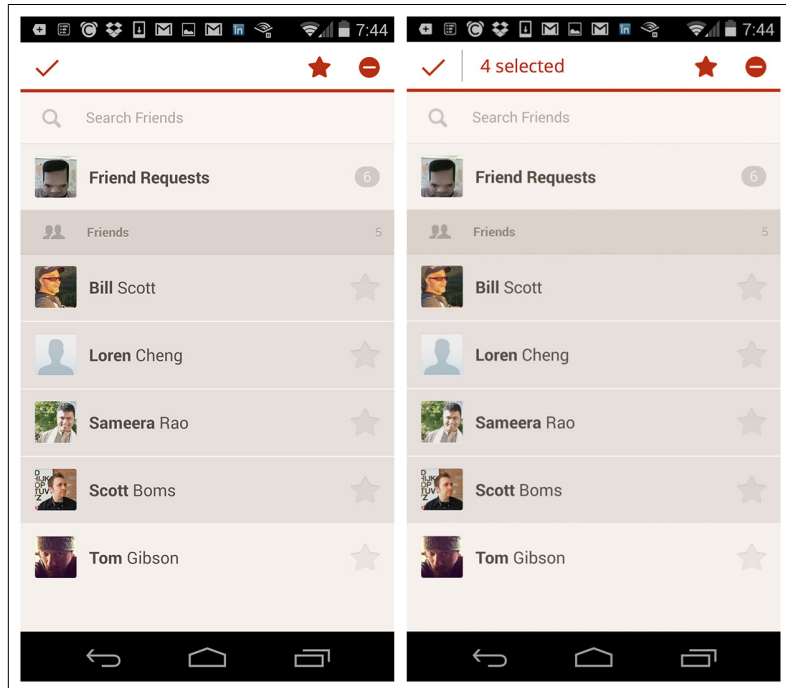


FIGURE 5-42.

Path for Android: current design on left; suggested improvement, showing selected number of items, on right

Lock Screen Controls

Both iOS and Android offer the option to display tools on the lock screen. This is seen most often in music apps, but not exclusively. If your application is running in the background and users may need quick access to pause, stop, or adjust it, consider designing tools for the lock screen.

The controls do not need to be limited to playing and pausing audio and video. Any.do for Android uses a widget to prompt app usage first thing in the morning, and also to help with tasks during the day.

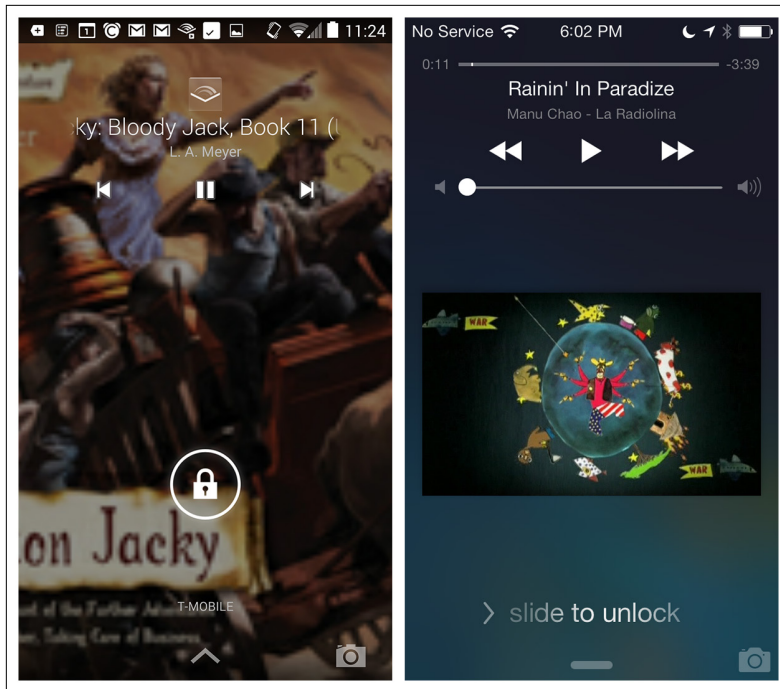


FIGURE 5-43.

Audible for Android and iTunes for iOS: lock screen controls

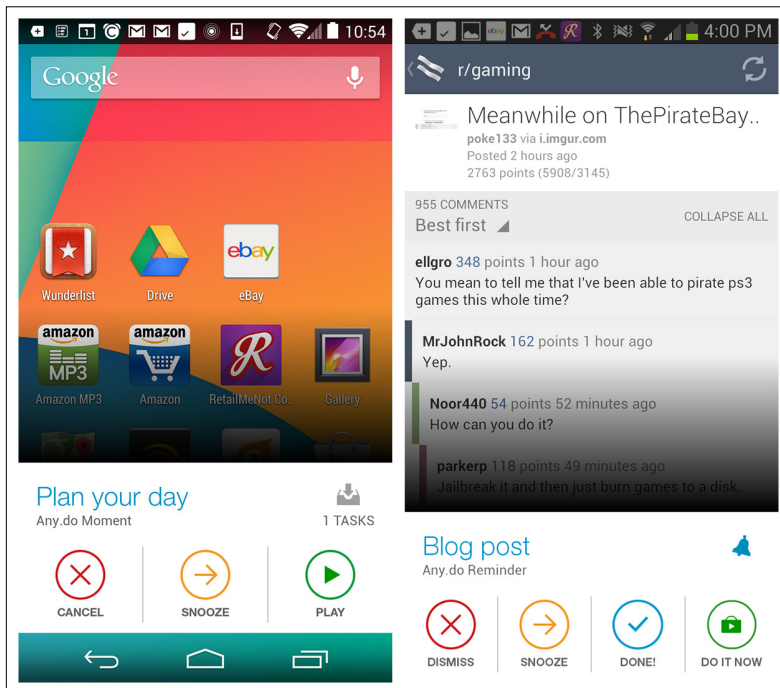


FIGURE 5-44.

Any.do for Android: widgets offer deeper engagement than notifications

[NOTE]

The mobile user experience may extend past the typical borders of the application. Consider lock screen controls or widgets to enhance the UX.